

ForkSCOPE

Supporting Information for

ForkSCOPE: Charting the Agentic Garden of Forking Paths

Anonymous Author(s)

This PDF file includes:

- Supporting text
- Figs. S1 to S14
- Tables S1 to S18
- SI References

Every number in this document is read from a released artifact. The vocabulary it describes is `ai+human.merged.forkmerged.merged.induced.r3.osplit`, recorded as `final_tag` in `study.json`, and each table names the file its figures come from. The pipeline, prompts, response cache and all intermediate artifacts are in the released package; file paths below are relative to its root. Section numbers of the form §4.3 refer to the main text; those of the form §S4 refer to this document.

Supporting Information Text

S1. Scope and Conventions

What this document carries. The main text argues for the method and reports the case study. This document carries what a reader needs to reproduce or dispute it: the corpora and their provenance (§S2), the build steps and prompts the main text refers to as Tables S3 and S4 (§S3, §S4), the parts of the normalization and evidence contracts that the main text states only in outline (§S5, §S6), the full reproducibility procedures and tables behind main text §6 (§S7 to §S10), the comparison against the corpus authors’ 21-slot schema (§S11), the scope conditions (§S12), the case-study decisions the main text refers to in main text §5.1 (§S13), and a reading guide to every view of the viewer (§S14). It does not restate the argument of the main text.

Terminology. We use the terms of Table 1 of the main text: *raw decision point*, *option* and *fork*. Two further terms are kept distinct throughout. An *analysis* is one analyst’s or agent’s work on the shared task, and a *replicate* is one re-run of our own pipeline. Variability across analyses is the object of study; variability across replicates is a property of the instrument.

Checking a reported figure. Each table names the artifact it was read from. A few figures are marked †: these come from the authors’ scoring records for an exercise whose inputs are not in the released package, and are reported as recorded rather than as recomputable. `scripts/trace.py` walks a fork to its options, an option to its raw decision points, and a raw decision point to a run and a line range; `scripts/compare_vocab.py` prints every vocabulary stage side by side. The two corpora are not redistributed (§S2), so the last step, from a line range to the literal source lines, resolves only when a corpus is mounted.

Headline counts. Table S1 extends Table 3 of the main text with the split by corpus and the concentration statistics. Two denominators recur: 317 forks is the whole induced vocabulary, while 41 is the number reached by at least 10 of the 223 analyses, and every concentration statistic is computed on those 41. The remaining 276 are reported as coverage rather than as agreement (§S12).

Table S1. The study at the headline vocabulary. Read from `data/vocabulary/bottom_up_...induced.r3.osplit.json`, `data/analysis/stability_...induced.r3.osplit.json` and `data/corpus/silent_decisions.tsv`. Every effective-option figure is `eff_all`, which gives one vote per analysis at its modal option and counts not reaching the fork as a level (§S9), averaged over the 41 measurable forks. The analysis count is corroborated by the call log: `01_segment_code` was called 223 times and `02_segment_prose` 234, the difference being the 11 human teams with a write-up but no deposited script.

analyses	223	forks	317
agentic	204	measurable (≥ 10 analyses)	41
human teams with a script	19	pivotal (<code>eff_all</code> > 1.5)	17
raw decision points	3,946	mean modal share, measurable	0.719
agentic	3,665	mean <code>eff_all</code> , measurable	1.75
human	281	median <code>eff_all</code> , measurable	1.37
options	777	silent decisions	333

S2. Corpora and Provenance

Acquisition. Neither corpus is generated by us or distributed with the package. The agentic corpus is the set of autonomous-analyst runs released by Bertran et al. [3]; the human corpus is the team material released with the many-analyst study of Silberzahn et al. [35], whose player–referee dyad dataset both corpora analyse. Table S2 gives the source, scope and artifacts of each. The agentic corpus is fetched by script under its authors’ terms. The human corpus is participant-supplied material from a human-subjects study, and acquiring it is a decision we leave to the reader against the original study’s terms rather than one a script makes on their behalf. Both are read from paths supplied in `FORKSCOPE_RAW_AI` and `FORKSCOPE_RAW_HUMAN`.

Verification. `data/raw/CHECKSUMS.txt` records a SHA-256 digest for each of the 697 artifacts the pipeline reads: 207 scripts, 206 reports, 207 transcripts, 76 human-corpus files and one dataset. `fetch_corpus.py` `verify` re-checks them. The check exists because an unfetched Git-LFS pointer is valid UTF-8 of roughly 130 bytes and is accepted downstream as a run whose report is empty, without any stage raising an error.

Held-out material. The agentic corpus ships with its authors’ own extraction against a fixed 21-slot schema, `decisions_mapped.json`. We hold it out of induction and use it only as a comparison set once the vocabulary ex-

Table S2. The two corpora. Artifact names are the literal values in `study.json:corpora`; counts are from `data/raw/inputs.json`. The agentic slice is one dataset, one base model and all five personas. Of its 207 workspaces, 206 have a report, one of those is empty, and the prose-length filter removes one more, leaving 204 runs, all of which contribute decisions. Of the 31 human teams, 19 deposited a script; extraction is code-anchored, so only those 19 contribute decisions, and the 11 further teams with a write-up were segmented but yield none.

	agentic	human
source	amazon-science/agentic-forking-path [3]	many-analyst study materials, OSF [35]
pinned at	commit d489f03, sparse checkout	released team directories
scope	claude-sonnet-4-5 on sample_soccer, 5 personas	31 teams
one run is	one workspace directory	one team-NN directory
code artifact	final_analysis.py (207 present)	the deposited script (19 present)
prose artifact	mirrored_report.txt (206 present, one empty)	larger of survey / report
runs charted	204 of 207	19 of 31
arm	one of five persona briefs	none
redistributable	yes, fetched by script	no, acquired by the reader

ists (§S11); the exclusion is enforced in `data/raw/inputs.json` rather than by convention. Its own record of what the schema could not hold is its `unmapped_decisions` list: a third of the 1,120 items its extractor recorded across the 207 runs (main text §5.1).

S3. Build Steps

Table S3 is the table the main text refers to as Table S3: the nine steps, the script behind each, and the artifact it writes. The whole build runs from one command:

```
python3 scripts/build_all.py --corpus ai,human --repair --gate
```

Steps 4a to 4d are the staged clustering of main text §4.3.2 and main text §4.3.3, and 4e is the repair loop; the loop is run as a loop rather than a sequence because uniting duplicate forks brings paraphrases that were held in separate forks into one fork, where they can be compared for the first time (Table S5).

Table S3. The build steps. Prompt identifiers refer to files in `prompts/`; see Table S4. Steps 4e and 4f carry the vocabulary from `.induced` to the headline tag: 4e iterates the option and fork merges to convergence, and 4f applies a deterministic option split that leaves every fork and every fork assignment unchanged, verified as an identical fork label set and an identical raw-decision-point-to-fork mapping across all 3,946 instances.

#	step	script	writes
1	distill	distill.py	data/distilled/, data/corpus/
2	cluster	cluster_up.py	data/vocabulary/bottom_up_<corpus>.json
3	audit and repair	audit_clusters.py, repair_clusters.py	data/audits/, split-only repairs
4a	merge options	merge_options.py	...merged.json
4b	merge forks	merge_forks.py	...forkmerged.json
4c	merge options again	merge_options.py	...merged.json
4d	induce forks	induce_forks.py	...induced.json
4e	repair loop to convergence	merge_options.py, merge_forks.py	...r1, ...r2, ...r3
4f	promote and split options	promote_repair.py	...r3.osplit, the headline tag
5	finalize	finalize_vocab.py	garden_<tag>.json, fork_stage_<tag>.json
6	analyse	stability.py, shapley_v2.py, iteration.py, ...	data/analysis/
7	audit the analyses	audit_garden.py, audit_shapley.py, ...	data/audits/
8	figures	build_figures.py and the per-figure scripts	figures/
9	validation gate	ablation.py, audit_ablation.py	data/ablation/, ablation.json

S4. Prompts

Table S4 is the table the main text refers to as Table S4. The cache key that pins each prompt is described in main text §4.2; here we add what the key does not show. `llm.py`, the only module that calls a model, distinguishes two failure kinds that require opposite responses: a transport failure is retried with the prompt unchanged, while a content failure, meaning a response that fails the schema or a semantic check, is retried with the specific complaint appended. An early version treated the two alike and lost 154 of 204 runs in one burst. `prune_cache.py` reports any cached answer that the shipped prompts can no longer reach, which is how the claim that the released data was produced by the released prompts is checked rather than asserted.

Table S4. The sixteen prompts the charting pipeline calls. Call counts and model assignment are read from the 2,273 records in `cache/llm_runs/`. The smaller model performs the mechanical passes and the larger model every pass whose answer changes the vocabulary. `prompts/` contains a seventeenth file, `30_silent_identity`, which the charting pipeline does not call; it is driven by `stability/scripts/q1c_adjudicate.py` for the semantic adjudication of silent decisions (§S7).

prompt	question it asks	caller	model	calls
<i>Extraction, one call per run over raw source</i>				
01_segment_code	partition a script into decisions and typed introns	distill.py	sonnet	223
02_segment_prose	partition a write-up into claims and non-claims	distill.py	sonnet	234
03_link	which claim describes which decision, and do they agree	distill.py	opus	223
<i>Typology, induced from items and never chosen in advance</i>				
10_induce_types	what categories are actually present in this sample	cluster.py	opus	7
11_assign_types	assign every item, NEW allowed and encouraged	cluster.py	sonnet	28
<i>Vocabulary: decisions to options to forks</i>				
12_group_batch	group decisions by the action they perform	cluster_up.py	sonnet	73
13_merge_groups	are these two groups the same action	cluster_up.py	sonnet	165
22_induce_forks	which options are rivals answering one question	induce_forks.py	opus	22
<i>Adjudication for repair: these produce verdicts, other scripts apply them</i>				
19_option_granularity	within one fork, which options are the same action	audit_granularity.py	opus	659
20_option_purity	do these decisions all belong under this option	audit_lineage.py	opus	110
21_fork_identity	are these two forks the same question	audit_lineage.py	opus	326
<i>Interpretation, small passes over the finished vocabulary</i>				
14_fork_stage	which lifecycle stage does this fork belong to	finalize_vocab.py	opus	24
15_fork_relation	how are these two forks related	garden.py	opus	4
16_align_channels	does this prose span describe this code decision	ablation.py	opus	162
17_fork_rationale	what assumption does this fork rest on	rationale.py	opus	9
18_merge_tags	fold these rationale tags together	rationale.py	opus	4

Code segmentation. The two hard requirements of `01_segment_code`, reproduced verbatim from the shipped file, are the coverage invariant of main text §4.3.1. Both are enforced in the retry path rather than checked after the fact.

1. Exhaustive. The union of all ``lines`` ranges across all spans must be exactly ``1..n_lines``, with no gaps. Blank lines count and must be assigned.
2. Exclusive. No line may appear in two spans. If a single line genuinely serves two decisions, assign it to the more specific one and add the other decision's id to ``shares_lines_with``.

The prompt also states an asymmetry for uncertain items: an item the model is unsure about is recorded as a decision with confidence: "low" rather than as intron, because a missed decision becomes silence in the record while a wrongly recorded one remains visible and can be removed later.

Stage assignment. `14_fork_stage` assigns each fork a stage of the data science life cycle [39, 40]. It receives the fork label and nothing else, for the reason given in main text §4.3.4. The blind assignment nonetheless recovers pipeline order, at Spearman +0.59 between assigned stage and observed script position (check I3, `data/audits/iteration_audit_...induced.r3.osplit.json`), which allows a departure from that order to be read as a departure rather than as labelling error.

S5. The Normalization Contract

The two rules of canonicalization, the foreclosure test, the structural veto and the density floor on fork merging are stated in main text §4.3.2 and main text §4.3.3. This section adds the worked cases that fix their reading, the prompt text of the test, and the trajectory of the repair loop.

Why operational behaviour, not wording. The skin-tone variable is the mean of two five-point ratings and so takes only multiples of 0.125. The thresholds > 0.125 and ≥ 0.25 therefore induce an identical partition of the sample, while > 0.25 and ≥ 0.25 are near-disjoint under an identical stated rationale: 27.7% against 58.2% of the sample classified as dark. Normalizing on the printed number would split real agreement and merge real disagreement at the same time.

Could one competent analyst, in one script, do both of these?

Yes: different decision points. Doing one leaves the other still open.

No, taking one means not taking the other: same decision point.

The foreclosure test, verbatim. *Drop rows with no rating* and *impute from the other rater* share no words and perform opposite operations, yet they form one fork, since taking either forecloses the other. *Drop rows with no rating* and *drop goalkeepers* are phrased almost identically, yet they form two forks, since one analyst performs both without contradiction. A lexical or embedding criterion misclassifies both cases, and the second misclassification is the more damaging, since it manufactures a fork no analyst faced.

Why the density floor is needed. Only a small fraction of all fork pairs is ever adjudicated, so most pairs inside a growing merge component have never been examined and no veto can fire on them. Unconstrained transitive closure would chain through those unexamined pairs; the 60% floor of main text §4.3.3 is what prevents it.

The repair chain and its stopping rule. Table S5 gives the trajectory. Option redundancy rises before it falls, because uniting duplicate forks brings previously separated paraphrases into a single fork where they can be compared. The loop stops when a round moves at most 2% of forks, and we report the round at which it stopped rather than claiming a fixed point, since at this threshold one need not exist.

The three rounds after `.induced` exist because of an ordering gap in the original pipeline. `induce_forks` re-derives fork labels from option purpose and ran after `merge_forks`, so any duplicate it created was invisible to the only pass that could unite it. The first fork merge after induction adjudicated 147 candidate pairs and judged 103 of them one question (`data/analysis/fork_merge_...induced.json`).

Table S5. The repair chain. Fork and option counts are read from the headers of the files in `data/vocabulary/`; option redundancy is measured by `audit_granularity.py` and is available for the stages shown. Step numbers refer to Table S3. `.osplit` is the deterministic option split applied after `.r3` and is the headline vocabulary; it leaves the fork layer unchanged.

step	vocabulary file (bottom_up_ai+human ...)	forks	options	moved	redundancy
2	<code>.json</code>	463	1,473	–	–
4a	<code>.merged</code>	463	1,112	–	–
4b	<code>.merged.forkmerged</code>	418	1,112	–	–
4d	<code>.merged.forkmerged.merged.induced</code>	375	966	–	0.776
4e	<code>.merged.forkmerged.merged.induced.r1</code>	329	774	12.3%	0.755
4e	<code>.merged.forkmerged.merged.induced.r2</code>	321	745	2.4%	0.937
4e	<code>.merged.forkmerged.merged.induced.r3</code>	317	729	1.2%	0.990
4f	<code>.merged.forkmerged.merged.induced.r3.osplit</code>	317	777	–	–

S6. The Evidence Contract and Check Verdicts

Enforced rules. main text §4.2 states the four outputs every check emits. Four rules are enforced on them in `scripts/exhibit.py`:

1. Exhibits are ordered worst-first. `exhibit()` requires a `worst` key function and sorts by it.
2. A REVIEW verdict with no exhibits raises an error.
3. A PASS requires a declared blind spot, supplied through `blind_to()`; an exhaustive check may declare that it has none, and the declaration must be explicit.
4. A PASS is refused while the check is entangled with the pipeline: `uses()` declares the evidence a check relies on and `BUILD_SIGNALS` names the signals the pipeline builds with. V1 and V3 rank on label similarity and V2 uses co-occurrence, which is a build signal, so none of the three can pass.

Rules 3 and 4 were each added after a failure. A token-overlap statistic reported 2.7% of within-fork option pairs as near-duplicates while two entire forks were duplicates that a reader found by comparing option lists directly; and an identity audit reported 27 of 30 label-similar fork pairs as one question when 17 of the 27 were pairs the repair had already refused to merge on co-occurrence evidence the audit does not observe, so it was restating a pipeline decision. The remaining 10 had never been adjudicated, and it was those that identified the ordering gap of §S5.

Reading an exhibit count. A count taken over a worst-first exhibit is conditioned on the selection that produced it. The 27 of 30 above is not a 90% duplicate rate: the 30 were selected as the most label-similar of roughly 70,000 pairs. We therefore report every such figure as a queue and its depth. A rate would require a random draw.

Table S6. Check-set verdicts, and the vocabulary each check ran on. Read directly from the JSON in `data/audits/` and `stability/data/audit_stability.json`, including the *ran on* column, which is the tag in the filename. REVIEW is a routed-to-review verdict rather than a failure; §S12 gives the reading of each. Numbering gaps are in the check sets themselves: no G3 or I4 verdict is written.

set	what it checks	written by	ran on	verdicts
I1–I6	iteration	<code>audit_iteration.py</code>	.osplit	I1, I2, I3, I5, I6 all pass
G1–G6	garden structure	<code>audit_garden.py</code>	.osplit	G1, G4 REVIEW ; G2, G5, G6 pass
H1–H6	attribution	<code>audit_shapley.py</code>	.osplit	H1 REVIEW ; H2–H6 pass
E1–E6	code ablation	<code>audit_ablation.py</code>	untagged	E2 REVIEW ; E1, E3–E6 pass
V1–V7	the vocabulary’s own exhibits	<code>audit_vocab.py</code>	.osplit	six REVIEW , V5 INFO
S1–S7	stability	<code>audit_stability.py</code>	.r3	S1, S5, S6 pass; S2, S3, S4, S7 REVIEW
granularity	option redundancy	<code>audit_granularity.py</code>	.r3	redundancy 0.990
lineage	option and fork identity	<code>audit_lineage.py</code>	.r3	2 of 13 pairs one question

Table S7. The V-checks on the headline vocabulary. Read from `data/audits/vocab_exhibits.json`. These checks rank items for review and are not used to draw conclusions. V2 is conclusive about the pattern it measures, though not about its cause.

	question	measurement	status
V1	within-fork option pairs that may be one action	561 of 5,611 above 0.55	queued, unread
V2	forks where one run took two options	43 of 317 , 386 pairs	REVIEW
V3	fork pairs that may be one question	24 of 50,086 above 0.60	queued, unread
V4	options bundling separable actions	116 of 777	REVIEW
V5	forks where nearly every run invented an option	0 flagged	INFO
V6	option groups spanning distant script regions	2 distant of 22 groups	REVIEW
V7	options with disjunctive labels	36 of 777	REVIEW

The single-choice assumption and its limits. The data model of main text §4.1 treats a fork as a slot at which a run picks one option and forecloses the others. V2 tests that assumption directly, and it does not hold universally: 43 of 317 forks contain runs that took two options, across 386 option pairs, of which 296 share exactly one run. Three causes produce the same signature and only inspection separates them. A genuinely multi-select fork shows many runs at a high rate; a declared robustness refit shows a single shared run, which is why most pairs share exactly one; and a clustering artifact shows few runs at a high rate. For forks in the first class modal share and effective-option counts are not well defined. We treat this as a limit of the model rather than as a clustering error to be repaired, and the policy file records the choice (§S13).

S7. Stability Under Replication

This section carries the procedures and tables behind main text §6.1 and main text §6.2.

Scope of the cache guarantee. The content-addressed cache establishes that a re-run reproduces the earlier answer, not that two independent builds would have agreed. Measuring the second property requires defeating the cache deliberately, and we report it as a separate exercise rather than as a property the pipeline asserts about itself. The forcing flag skips the cache read but still writes to the same key, so a replicate run in place would overwrite the baseline while measuring it. Every replicate therefore runs under a separate study root with an empty cache, so that every call misses and the baseline is unreachable by filesystem layout rather than by remembering a flag. The baseline cache was verified unchanged throughout, at 2,274 entries during that exercise; the released cache has since grown to 2,293.

Estimators. Both builds partition the same 3,946 raw decision points, keyed on (`run`, `line`, `text-prefix`) and verified identical across builds, so agreement at the option layer is exact arithmetic over two labelings of one item set. Matching option labels by string similarity would reintroduce the failure the pipeline rejects for clustering: 3,739 of 3,946 raw texts are distinct (94.8%) and 776 of 777 option labels remain distinct after canonicalization, so a lexical match rate would measure phrasing. We report the adjusted Rand index [17] (chance-corrected pairwise co-clustering), variation of information (a metric on partitions, needed once more than two builds are compared) and B-cubed precision and recall (per-instance, so precision above recall indicates that the second build split what the first held together). The direction of

disagreement is derived from two independent signals, the B-cubed asymmetry and the raw cluster count, and reported as AMBIGUOUS when they disagree.

Distillation. Thirty runs were distilled twice (Table S8). The three matching rules are reported separately because the level of agreement depends on which is used, and the lenient rule is the one the policy adopts (§S13). Silent decisions reproduce at F1 0.489. A semantic adjudication, blinded to build order and permitted to match on meaning rather than line overlap, returned 0.4889 against a structural 0.4889, so the disagreement is neither boundary jitter nor an artifact of the matching rule: the two builds flag genuinely different operations as unmentioned. The aggregate result, that analyses under-report themselves and by roughly how much, is stable across builds (§S8). A claim that a particular decision in a particular run went unmentioned is not.

Table S8. Distillation agreement across two independent builds. Thirty runs distilled twice under a separate study root. Read from `stability/data/q1_distill.json`, `q1b_residue.json` and `q1c_semantic.json`. The lenient and core rows are the two ends of the *decisions* range in Table 5 of the main text.

level	matching rule	F1
verbatim	identical line sets	0.176
lenient	Jaccard ≥ 0.5	0.713
core	containment ≥ 0.5	0.895
median boundary jitter among matched pairs		1 line
code exon/intron typing agreement		0.886
prose spans (ARI), the least stable thing measured		0.480
silent decisions (F1), structural and semantic		0.489

Vocabulary. A vocabulary replicate copies data/distilled into the isolated root, so its input is byte-identical to the baseline's and every downstream difference is vocabulary variability alone. The two builds recovered 317 and 351 forks over 729 and 759 options. Table S9 gives the agreement at each layer.

Table S9. Vocabulary agreement, measured over the items each layer actually clusters. Read from `stability/data/layer_agreement.json`, `per_run_*.json`, `forks_that_matter.json` and `audit_stability.json`. The replicate was run at `.r3`, one step before the headline tag; only the last row is at `.r3.osplit`. The options row is the *options* row of Table 5 in the main text, and the lenient fork-membership figure is its *membership agreement*. The gap between the two fork rows reflects option size rather than method: 497 of the 777 headline options (64.0%) are singletons held by one analysis and 215 of the 317 forks hold a single option, so an unweighted fork agreement is dominated by the placement of rare options while the instance-weighted figure is dominated by the options that carry the corpus.

layer	items	agreement
options over decisions, ARI	3,946	0.853
forks over options, unweighted ARI	497	0.3065
forks over options, instance-weighted ARI		0.8522
fork membership, lenient		0.47
(<i>confounded</i>) forks over decisions, ARI	3,946	0.695
per-run option grouping, median ARI	223 runs	1.000
per-run fork grouping, median ARI	223 runs	0.590
pivotal forks, A against B	10 qualifying	0.975

Repair against a contraction null. Repair contracts the vocabulary, from 463 to 317 forks in one build and from 489 to 351 in the other, and contraction changes agreement mechanically. We therefore compare it against a null that merges each base vocabulary at random to the repaired granularity, matching the observed cluster-size distribution. Option ARI moves from 0.646 to 0.853 against a null of 0.187 ± 0.014 ($z = 46.8$), and fork ARI from 0.565 to 0.695 against 0.219 ± 0.004 ($z = 122.0$); both fall below the empirical p floor of 0.005 at 200 draws. The confound runs against the result: random contraction reduces agreement between builds, from 0.565 to 0.219, so repair overcomes a mechanism that would otherwise cost roughly a third of the agreement (`stability/data/repair_null_option.json`, `repair_null_fork.json`).

The trunk converges; the tail does not. Conditioned on coverage, the two builds agree exactly: 40 of 40 forks reached by at least 11 runs match across builds, and 18 of 18 at coverage of at least 25%. The entire divergence between 317 and 351 lives in the rare tail, which the 10-run floor of §S12 excludes from every reported statistic. The floor, the

fork-matching threshold and the coverage gate were each introduced for independent analytical reasons, and each selects the population over which the pipeline reproduces best; the reported analyses are therefore better supported than the whole-vocabulary figures indicate, and we report both facts together so that the second qualifies the first.

Run geometry. Every reported analysis consumes run-level structure, meaning which forks a run touched, rather than fork identity. Encoding each run as the set of clusters it touches and comparing the two builds’ run-by-run Jaccard matrices, the Mantel correlation [25] is $r = 0.766$ at the fork level and 0.857 at the option level, against a permutation null with mean zero ($z = 15.6$ and 17.7; `stability/data/run_geometry.json`). Under the conservative construction, restricted to clusters of at least five runs with run size partialled out, the figures are 0.632 and 0.762; the main text reports the fork-level range as 0.63 to 0.77. Weighting the sets by decision count moves the fork figure by +0.025 and the two geometries correlate at 0.968, so the binary representation loses almost nothing.

Isolating fork induction. Builds A and B agree well about options and badly about forks, and two readings fit that equally: induction is unstable in itself, or it is near-deterministic and amplifies an options layer that already differed. Replicates that freeze the options and re-run only fork formation separate them. Five replicates were run against each parent’s frozen option layer, a count pre-committed before the first build (§S13), and within one parent the comparison is exact: every replicate labels the same options file and the chain never rennumbers an option, so it is a partition comparison with no matching step. Table S10 gives the result. Induction disagrees with itself at ARI 0.62 on byte-identical input. Decomposed as shortfall from perfect agreement, induction contributes 0.384 and the differing options 0.282, so induction is the larger share at 58%, and a perfectly stable options layer would still land at 0.62. The fork count is not stable either: the five replicates of parent A returned 299, 281, 302, 317 and 297 forks and those of B 298, 317, 314, 311 and 303, so 317 is one draw from a spread of roughly 281 to 317 on fixed input, and the parents’ 317 and 351 are about two standard deviations apart rather than a structural difference.

Two qualifications. The replicates drop `merge_options` from the repair loop, so they measure fork-formation variance under frozen options rather than end-to-end variance, and the parents’ own fork layers, built by the full loop, are excluded from every statistic. And the replicate vocabularies were built under an external study root and are not in the released package; the figures are recorded in `stability/NOTES.md` and `report/REPORT.md`, and `stability/scripts/fork_variance.py` recomputes them from a replicate root.

Table S10. Fork induction under frozen options. Ten replicates, five per parent, of fork formation only. Within-parent pairs compare two labelings of one option set; between-parent pairs require option matching and inherit its caveats, so the two are not averaged. Recorded in `stability/NOTES.md` and `report/REPORT.md`; recomputed by `stability/scripts/fork_variance.py`.

	pairs	ARI	sd	df
within parent A	10	0.595	0.071	9
within parent B	10	0.636	0.047	9
between parents	25	0.334	0.036	–

Specific findings, re-derived independently. Of 225 drawn highway routes, 180 could not be compared because an endpoint fork had no confident match; route matching succeeds on well-populated forks and fails on the rare tail, so the reproducibility of those 180 is unmeasured rather than low. Among the 42 comparable routes, traffic rank correlation is $\rho = 0.951$ against a permutation null with mean 0.001 ($z = 6.08$; `stability/data/q4_replicate_r3.json`); stratified by traffic the correlation is 0.904 on the low-traffic half and 0.880 on the high-traffic half[†], so it is not carried by a few large edges. Per-cell persona enrichment does not reproduce, at 1 of 10, whereas the omnibus test of whether runs sharing a persona walk more similar routes is positive in both builds, at +11.2% with $z = 10.25$ and +11.9% with $z = 10.46$ (`persona_assoc_A.json`, `persona_assoc_B.json`). We therefore report that personas walk different routes and treat the enriched cells as scan output, since which particular edge carries the signal is not identifiable at this sample size.

Scope comparison, kept separate from replication. This is the comparison main text §5.2 reports. Comparing the pooled vocabulary against a build over the agentic corpus alone confounds run-to-run variability with the effect of adding the 19 human analyses, so it is reported apart from the replicate figures above. Option ARI is 0.865 and fork ARI 0.756 (`stability/data/q3_scope_pooled_vs_aionly.json`). Even there, highway structure is more stable than the vocabulary that carries it: only 29% of forks match strongly across the two builds, yet among the 64 routes present in both, traffic rank correlation is 0.928 (`q4_scope_pooled_vs_aionly.json`).

Provenance of runs. Runs added to the garden after the initial build carry one of three provenance tags (`vocab.py:PROVENANCE`): `corpus` for a run extracted from an analysis an analyst or agent wrote, `itinerary` for one assembled from options the corpus already contains (main text §4.4, `/itinerary`), and `authored` for one containing an operation no corpus run performed. Non-corpus runs are counted rather than excluded, and every concentration statistic can be reported with and without them, so a headline that moves with the provenance mix is visible rather than silent. Every run in the case study is `corpus`.

Observed hazards. main text §6.1 names two, and main text §7.2 records that both are now mitigated. First, the cache key records a model alias rather than a resolved version, so an alias repointed to a newer model would let old entries hit while new calls were served by a different model, with nothing in the output recording the mix. The remedy is that the resolved model and client version are recorded at call time (`model_resolved` and `cli_version` on every entry and log line). The entries of the first build predate that fix and carry no resolved version, so the run-to-run figures above are upper bounds on same-model variability: some part of the measured drift may be version drift, and the first build cannot separate the two. Second, a run’s persona arm is a property of where the run lives in the raw corpus, and for records written before the field existed it was recovered by re-reading that directory. The raw corpora are fetched rather than vendored, so the directory can be absent, and when it was the recovery silently fell back and produced a spurious extra arm of 43 runs[†] that belonged to one of the five personas; totals were conserved exactly, so only the per-arm attribution was wrong. The remedy is that the arm is repaired at the source record and a failed recovery raises an error rather than falling back; `fetch_corpus.py` verify before a build and `stability/scripts/replicate.py`’s preflight refuse the degraded state as well.

A pinned-model cross-check. A separate pilot build over the agentic corpus alone, exploring an agent-driven review workflow that is otherwise out of scope, pinned its reader model to `claude-sonnet-4-5-20250929`, the snapshot that generated the corpus, on every one of its 954 calls[†]. On a 30-run stratified replicate of that build the decision layer reproduces at lenient F1 0.804 (precision 0.796, recall 0.813; verbatim 0.210 and core 0.904 bracket it)[†], above the 0.713 of Table S8. This is directionally consistent with the first hazard: some of the unpinned build’s lower reproducibility may be model drift rather than the extraction task. It is a directional cross-check and not a corrected figure, for two reasons. The sample is small, 30 of 207 runs with four degrees of freedom after stratification. And because the reader model there is the snapshot that wrote the corpus, higher self-agreement is also consistent with a model recognising its own output, a confound the unpinned build does not share.

S8. Recovery of Decisions from Prose Alone

Extraction is code-anchored, so 12 of the 31 human teams contribute no decisions. Whether prose-only teams may be admitted gates any pooled comparison, and gate 5.4 tests it directly: each run’s prose is segmented blind to its own code and matched against the code channel by prompt 16, over a stratified sample of 80 runs. Table S11 gives the result and its calibration.

Table S11. Code ablation, gate 5.4, overall and by corpus. The *all* column and the null are the gate verdict in `data/audits/ablation.json`. The per-corpus columns are the figures main text §5.1 reports, from the second run of the alignment judgement described below, recorded in `report/REPORT.md`. Recall is the share of code decisions that prose also found; agreement is the share of matched pairs judged the same option; vague is the share where the prose is too underspecified to pin the option. The null pairs run *i*’s code against run *j*’s prose.

	all	agentic	human
runs sampled	80	61	19
code decisions / prose spans	1,363 / 3,329		
matched pairs	808		
recall	0.593	0.642	0.534
agreement on the option, strict	0.952	0.921	0.847
agreement on the option, lenient	0.984		
vague	0.032	0.063	0.073
gate threshold	0.85		
null , spurious match rate across unrelated runs	0.489		

Agreement clears the threshold, and we do not read that as a pass. The matcher pairs unrelated runs at 82% of the within-run rate, because every run analyses the same dataset and makes many of the same decisions, so a cross-run match is frequently a real correspondence and not evidence that prose recovered the choice made in that run. What survives calibration is agreement conditional on matching, 0.952 within-run against 0.649 in the cross-run null. The supported

conclusion is narrower than the headline: prose is accurate where it speaks and often silent. 40.7% of code decisions are never mentioned in the sampled runs. Silent decisions, code with no claim describing it, are 7 to 9% of decisions (main text §5.1): 0.092 of decisions in one build and 0.072 in a replicate, and 333 of the 3,946 raw decision points in the released corpus (8.4%; `data/corpus/silent_decisions.tsv`). The rate is model-dependent, since it is recovered from code–prose linking: main text §7.2 reports that switching the reader model roughly doubled it, and validation for this task is limited to annotator agreement. The human corpus is weaker than the agentic arm on recall and on agreement, and on its own it falls below the 0.85 gate, which is why main text §7.2 licenses no outcome-level contrast between the corpora. Its conflicts are outright disagreements between script and write-up, including a 0.75 against a 0.625 cutpoint, logit against probit, and a one-sided against a two-sided p .

Stability of the alignment judgement. Prompt 16 was called twice on the identical 80-run sample, five days apart, with no change to corpus, code or prompt. Agreement conditioned on both runs matching was 0.872 and stability over the union of matches 0.796 (check S5). Verdict transitions were asymmetric: 25 pairs moved from `same_option` to `vaguer` and 8 to `same_fork_different_option`, against 7 moving back, a shift in severity rather than symmetric noise. Downstream, strict agreement fell from 0.9517 to 0.9077 and check E4 moved from PASS to REVIEW. The per-corpus columns of Table S11 are from that second run, which is the one the main text reports; the *all* column and the null are from the first, whose verdicts are the ones written to `ablation.json`.

S9. The Attribution Surrogate

Pivotal forks. The effective number of options is the inverse Herfindahl–Hirschman index over option shares [14, 16], $1/\sum \text{share}^2$. A fork is pivotal when it meets both a coverage floor, at least 25% of the 223 analyses reach it, and an effective-options floor, above 1.5 among those that do (main text §5.4). Eighteen forks clear the coverage floor and 17 of them also clear the effective-options floor (main text §6.2). The viewer computes the same selection as one threshold on `eff_all`, which counts not reaching the fork as a level so that low coverage lets the absent level dominate and agreement lets the modal option dominate (`vocab.py:is_pivotal`); its Pivotal Forks tab exposes both the threshold and the coverage floor as controls (§S14).

Design. Attribution is a closed-form Shapley decomposition [34] over a surrogate fitted to reported outcomes rather than by re-execution, so the scores inherit surrogate error. The surrogate is a ridge regression on one-hot level indicators with the penalty cross-validated per payout over $\{1, 3, 10, 30, 100, 300, 1000\}$, since on a design this wide relative to 204 outcome-bearing runs an unregularized fit is not well posed. A level requires at least 10 supporting runs to enter the design and a fork at least two such levels to enter as a player; not reaching a fork is itself a level, since requiring every run to reach every fork would discard the most common thing a run does at most forks. This is the expanded set of 36 players of main text §5.4, and Figure 12 of the main text shows its verdict table in the viewer. The bootstrap uses 2,000 draws at a recorded seed.

Additivity. The surrogate cannot represent an interaction of the form *dichotomizing matters only in combination with clustering*, so whether additivity costs anything was tested directly, against gradient-boosted stumps (additive by construction), depth-3 trees and a random forest, on the same folds and seed over the 17 pivotal players (Table S12). Allowing depth makes out-of-sample prediction worse on every payout, substantially so for $\log(\text{OR})$, and we retain the additive form.

Table S12. Interaction gain: out-of-sample R^2 by model class, on the 17 pivotal players. Read from `stability/data/surrogate_form.json`; five folds, $n = 204$. Gain is the best non-additive model against ridge; negative means depth hurts.

payout	ridge	stumps	depth 3	forest	gain
verdict	0.262	0.213	0.064	0.254	−0.149
$\log(\text{OR})$	0.032	−0.079	−0.636	−0.028	−0.557
z	0.246	0.236	0.114	0.209	−0.122
CI width	0.046	−0.018	−0.075	−0.015	−0.057

Reading Table S13. The forks predict the reported conclusion and its z statistic out of sample, and predict the effect size poorly: the pivotal-only fit on $\log(\text{OR})$ is 0.032. Adding novelty as a player raises the fit on the two effect-size payouts and leaves the two conclusion payouts almost unchanged, the pattern expected if novelty acts on the magnitude rather than on the verdict (§S14). Restricting the player set to the 17 pivotal forks reduces the fit on every payout, which

Table S13. Cross-validated R^2 of the attribution surrogate, by payout and by player set. Read from `data/analysis/shapley_*.json`. The `.r3.osplit` and `pivotal only` columns are the two columns of Table 4 in the main text, which calls the 36-player set the expanded, or contrast, set. The leftmost column is the vocabulary before the deterministic option split, on which the surrogate does not predict out of sample.

payout	.r3 36 players	.r3.osplit 36 players	+novelty 37 players	pivotal only 17 players
binary verdict	−0.009	0.293	0.297	0.262
z statistic	−0.012	0.307	0.320	0.246
log(OR)	−0.008	0.081	0.114	0.032
CI width	0.001	0.038	0.082	0.046

measures how much of the explained variance lies outside the forks a triage on coverage and contestedness would surface. The bootstrap interval on the Shapley strength field is computed over a mean of absolute values and cannot cover zero for any data; it is not a test of the null, and a permutation null replaces it wherever a significance claim is made (check H2).

S10. Human Benchmark

A self-agreement rate is not interpretable in isolation: 0.70 is poor against raters who agree at 0.95 and at ceiling against raters who agree at 0.70. The benchmark of main text §6.3 supplies the comparison; this section gives the protocol, the scoring rule and the per-task results. The packet, the instructions given to raters, the key and the scorer are in `reviews/human-benchmark/` and `stability/scripts/score_review.py`. The returned sheets are held by the authors and are not in the released package, so the results below are marked †.

Items and order. Three raters, who are authors of this paper as main text §6.3 states, answer the same items independently, in a fixed order, without seeing the pipeline’s answers; the blinding controls for seeing the model’s specific verdicts, not for familiarity with the study design, which is why the packet fixes the questions and the strata in advance. Part 1 presents 43 pairs of raw decision points and asks whether they describe the same operation on data, however differently worded. Part 2 presents 60 pairs of options and asks whether they are alternatives at the same fork, decided by the foreclosure test of §S5. Part 3 presents 30 whole forks with their options and counts and asks whether each is over-merged, under-merged, correct or unsure. The order is enforced because Part 3 reveals how the pipeline grouped the material and would otherwise supply Parts 1 and 2 with context the model did not have. *unsure* is available at every part, and an item every rater marks *unsure* is a result rather than a missing response. Confidence is recorded alongside each verdict. Items are drawn from the `.r3` vocabulary, one step before the headline tag, so they characterise the pipeline’s self-agreement at each layer rather than the exact reported vocabulary. Each pair carries the verdict of both builds, and items were stratified by whether the builds agreed on the pair (*stable same*, *stable different*) or disagreed (*unstable*); every item a rater saw came from one build only, and the second was used solely to sort items into strata. Each rater’s sheet was shuffled independently so row order could not create a common anchor.

Scoring rule. Each pair already carries two model verdicts, since build A either grouped the items or did not and so did build B, and that binary is the one the rater answers. Model–model and human–human agreement are therefore computed on identical items with identical response options. Human agreement is the mean pairwise agreement across raters, not majority-versus-individual agreement, because with three raters the majority is right by construction and the figure drifts upward with rater count; pairwise is what the two builds give, so pairwise is what the raters must give. A sheet that misses a gold pair is reported separately and excluded from the headline rather than dropped.

Registered reading. The hypothesis is recorded in the packet before any rating. If raters also split on the pairs the two builds disagree about, the pipeline’s instability tracks genuine ambiguity in the task; if raters agree confidently on those pairs, the instability is model noise and indicates the prompt rather than the task. Both outcomes are reportable and the packet says so in advance. The policy file (§S13) names this benchmark as the only external criterion against which the stability figures of §S7 may be read, and records that no tuning toward higher self-agreement is authorised.

Part 1: the options layer is at the level of human agreement. Reweighting each stratum by its share of the full candidate pool, two raters agree with each other at 0.818 and two pipeline builds at 0.849 (Table S14)†. Two pipeline runs agree slightly more often than two people do on the same items, so there is no headroom left at this layer: making the pipeline more self-consistent would only make it more consistent than people are.

Table S14. Pool-weighted agreement at the options layer.[†] Mean pairwise agreement across the three raters, against the agreement of the two builds on the same items.

agreement over the pool	
two raters	0.818
two pipeline builds	0.849

Part 2: where the pipeline agrees with itself, it is also right. Taking the human majority as the reference on the pairs where both builds returned the same verdict (Table S15)[†]: every one of the 19 pairs both builds placed at the same fork, the human majority also grouped, and it holds at every level of stringency (19 of 19 with one rater committed, 17 of 17 with two, 15 of 15 with all three). This is a validity check a second pipeline run alone cannot supply, since two runs making the same mistake agree perfectly with each other.

Table S15. Human-majority endorsement of stable pipeline verdicts.[†] Exact binomial p against chance.

what both builds did	pairs	majority agreed	binomial p
grouped them at one fork	19	1.000	< 0.0001
kept them apart	18	0.833	0.008

Part 3: some forks are two questions under one label. The free-text instructions converge on one finding: 8 of 14 commented forks independently name the same confusion, that the vocabulary conflates *which variables enter the model* with *how each variable is transformed before it enters*[†]. Six forks carry this tangle, from the most contested fork in the corpus (*which covariates make up the model's adjustment set*) down to a fork over continuous-predictor transforms. Two raters using Part 3 alone flagged the same fork as under-merged, and two option pairs both builds had kept apart were unanimously called rivals at one fork by all three raters; both pairs straddle the same two forks a rater's free text had already named. The composite-fork problem therefore recurs, is nameable, and raters converge on where it is without seeing each other's answers. Endorsement, whether a rater accepts the fork as posed, runs well ahead of exact-verdict agreement (0.711 against 0.511)[†]: raters largely agree the map is broadly right and disagree on how to describe what is wrong with the rest. The implication for the tool, taken up in main text §6.3, is a review primitive that splits a fork along a reviewer-named axis rather than only merging it or leaving it.

What the benchmark does not settle. At the options layer, raters agree less with each other on the pairs the two builds disputed than on the pairs they agreed about (0.641 against 0.782)[†], a gap the registered thresholds place in an undeclared middle rather than a clear reading. At the fork-pair layer the opposite holds: raters agree with each other on disputed pairs about as readily as on settled ones (0.754 against 0.750), within the registered band for *instability is not tracking difficulty*, while the pipeline's agreement with the human majority falls to chance (0.450 to 0.462) on exactly those pairs[†]. Read plainly, the pipeline's fork-pair instability is not standing in for genuine ambiguity in the item: people find these pairs easy where the builds do not agree with each other or with people. The inferential form of the test, the AUC of a rater-derived difficulty score against the disputed/undisputed split, does not reach significance at this sample size (AUC 0.499 to 0.632, $p > 0.08$ in every reading)[†], so neither direction is overstated; what the descriptive comparison argues for is a revised foreclosure prompt or more fork-pair training signal.

Limits. The comparison is harder on the raters than on the pipeline: a rater sees two sentences, where the pipeline's adjudication step sees roughly 45 candidate options at once together with a co-occurrence table, so every human figure here is a floor on what a person given the pipeline's own context would achieve. With three raters on identical items an agreement rate such as 0.833 means 5 of 6 rater pairs and its uncertainty is wide, and nothing here generalises to a different rater pool. The two figures compared in main text §6.3 are not the same statistic as the fork-induction ARI of Table S10: one is a pairwise same/different rate over sampled pairs, the other a partition comparison over a frozen-options replicate, and the claim is that they are consistent in direction and magnitude, not that one recomputes the other.

S11. Comparison Against the Corpus Authors' Codebook

Bertran et al. release, alongside every run, their own extraction of that run's decisions against a fixed codebook, `decisions_mapped.json`, built per dataset by model-driven compression over a larger candidate set. The main text cites its headline number in §2.3 and §5.1: a third of extracted items left unmapped (§S2). Here the induced vocabulary is compared

against it directly, in two ways. The file was never read as pipeline input: it was compared only after both maps existed, the study manifest names two input artifacts, every distillation record shows only those, and the response cache contains no call that saw it. The file declares 21 slot names; 10 of them partition the runs and enter the structural comparison, and the curated codebook for this dataset that the adjudicated comparison was run against has 11 nodes.

Structural comparison. Both vocabularies partition the same 207 runs: their slot assigns each run a `choice_made` and our fork assigns it an option. A slot and a fork are the same question to the extent that those partitions agree, and no labels are read, so a slot and a fork that share no vocabulary can still match (`stability/data/aws_compare.json`, computed at `.r3`). Two estimators were rejected before the reported one. Adjusted Rand index rated five constant slots as in perfect agreement with one unrelated constant fork, since both are the trivial one-cluster partition; we require at least two distinct values on each side. Raw normalized mutual information approached 1 for any pair with 13 to 20 shared runs and 5 to 12 categories a side, where nearly every run occupies its own cell, and returned 0.73 between a transformation slot and a fork about how the conclusion is worded, at ARI -0.02 ; we use adjusted mutual information with a 30-run floor. Against the 22 of our forks reached by at least 30 of their runs, 7 of the 10 real slots are recovered at $AMI \geq 0.15$, and reading the labels gives 6 clearly recovered, 2 partial and 2 not. The statistic understates the correspondence: the most obvious match, `skin_tone_operationalization` against *how is the skin-tone rating entered as the exposure*, scores AMI 0.11 because their slot has 32 distinct choices over 203 runs against our 17 options, and two partitions of one question at very different granularity agree poorly under any chance-corrected index.

Node-for-node adjudication. The second comparison shortlists candidate pairs lexically and puts each to the same instrument the pipeline uses to judge fork identity within a build, prompt 21. Its inputs are not in the released package, so its figures are marked $\dagger$. Of 312 candidate pairs from the lexical pre-screen, 278 were adjudicated at high confidence, and 10 of the 11 curated nodes (91%) have a counterpart among the induced forks $\dagger$. This is a floor rather than an exact figure: the lexical screen is itself uninformative about fork identity on this comparison, with matched pairs at median cosine similarity 0.28 against 0.27 for non-matches $\dagger$, so a curated node whose true counterpart shares no vocabulary with it never reaches the adjudicator.

Symmetrically, 15 of the 25 most-walked induced forks (60%, accounting for half of the run-appearances among that top 25) have no counterpart anywhere in the curated codebook $\dagger$. These are not tail cases: the most common is the data-provenance fork, *which version of the source dataset is loaded*, walked by 94% of the corpus; others include which effect-size measure is reported and how missing skin-tone ratings are handled. Their own `unmapped_decisions` tell the same story, and include *choice of estimand*, *adjusted risk difference rather than odds ratio* and *specification search for optimal skin tone threshold: tested 6 thresholds*, both of which the induced vocabulary holds as forks.

The one node that did not match. `sample_definition.exclusion_criteria`, the curated node with the most distinct option strings (80, more than twice any other), matched none of 30 adjudicated candidates $\dagger$. Its options are conjunctions, such as *exclude missing covariates plus exclude medium skin tone*, rather than alternatives, and the adjudicator's stated reason for declining every candidate was that the node spans several independent exclusion decisions, missing skin tone, missing covariates, goalkeepers and league, that the induced map keeps as separate forks. We read this as the induced map correctly declining to represent a bundle as one choice, the same composite signature the human benchmark surfaces independently (§S10), rather than as a failure to match. Table S16 shows the underlying difference in kind: *did not reach* has no representation in the curated map, where every assignment carries a substantive value, whereas most induced forks are reached by few analyses.

Table S16. The curated and induced maps are not the same kind of object. Induced counts are from `data/vocabulary/bottom_up_...induced.r3.osplit.json`; curated counts $\dagger$ from the adjudicated comparison. Every one of the curated map's 2,268 assignments carries a substantive value; *did not reach* does not exist in it.

	curated	induced
forks	11	317
reached by ≥ 206 analyses	11	3
reached by ≥ 10 analyses	11	41
reached by exactly 1 analysis	0	169
median reach (analyses)	206	1

An error caught and fixed. The first run of the adjudicated comparison returned recovery of only 2 of 11 nodes $\dagger$. The curated labels are `snake_case` identifiers and the induced labels are natural-language questions; tokenized as written,

the two sides shared no vocabulary, every cosine similarity was exactly zero, and the matching silently fell back to alphabetical order rather than failing loudly. Splitting identifiers into words and including each fork's own option text as matching context raised the non-zero-cosine share from 0% to 39% and moved recovery from 2 to 10⁺. We report this because the same failure mode, a similarity screen collapsing silently rather than visibly, threatens any cross-instrument comparison built this way.

Scope. Neither comparison is a validation. A matched pair shows that two model-produced maps found the same question, not that either is correct, and the adjudicator is a model from the same family as both maps. The released file carries per-slot confidence and evidence fields characteristic of a model-generated extraction, and the corresponding supporting information does not state who authored the schema, so recovering another model's extraction is the only claim supported here. The charting layer behind the adjudicated figures had, at the time of that comparison, not yet been settled by this study's own gate, and fork-level reliability had not cleared its audit check (§S6); none of the three findings above depends on that layer, since they rest on counts and adjudications, but we note the status rather than omit it.

S12. Scope Conditions

Coverage. Concentration measured over four analyses is not informative, so only forks reached by at least 10 of the 223 analyses enter a stability statistic, which is 41 of 317. The remaining 276 are reported as coverage rather than as agreement. The same floor governs the surrogate of §S9.

Observational design. Persona arm is assigned by the corpus authors and no other variable is assigned. A persona brief plausibly influences both what an analysis does and what it concludes, so every relation between a fork and an outcome is reported as an association.

Permutation resolution under multiplicity. A permutation p cannot fall below $1/(B+1)$. Under Benjamini–Hochberg correction over many tests the corrected threshold frequently lies below the floor of an inexpensive screen, so a test can return no significant result by construction. Every permutation test reported here escalates its draw count for anything that survives a screen, and we quote the empirical floor rather than a point estimate of p where the two coincide.

The REVIEW verdicts. G4 identifies candidate conditional gates, but a reach-preserving permutation produces a comparable number from noise, and most candidate triples fail because fewer than five analyses walked the path; at this corpus size a flat decision space and an underpowered test are not separable, and we claim no conditional structure. G1 screens 21 of 43 label-similar fork pairs and excludes 22 on co-occurrence, which is the appropriate signal for a single-choice slot and not for the multi-select forks of §S6, so the excluded pairs are carried as a queue. E2 is the ablation null of §S8. H1 flags the gap between the surrogate's in-sample and cross-validated R^2 , which is why only the cross-validated figures of Table S13 are reported. The S-checks carry fixed thresholds that encode a decision, set once and not moved to make a check pass: S2 routes the option layer to review because its B-cubed F1 of 0.759 falls below 0.80, S3 the fork layer because its ARI of 0.695 falls below 0.75, and S7 the residue because silent decisions reproduce at F1 0.489 against a 0.70 floor, so their counts must carry uncertainty rather than be quoted as point estimates. S4 reports 379 unstable option clusters (52.1%), whose pairs default to unmerged. The V1 and V3 queues are ranked and unread, and are reported as queue depths.

What is and is not claimed. Supported: that analyses under-report their own decisions, and roughly how often; that the corpus has this many broadly faced forks; that the pivotal forks predict the reported verdict out of sample; that personas walk different routes. Not supported: that a particular decision in a particular run went unstated; that a named fork has N options or effective options X ; that a particular highway cell is persona-enriched; and, until both corpora clear the gate independently, any outcome-level contrast between the agentic and human corpora (main text §7.2); the structural comparison of main text §5.2 is the narrower claim that survives. Under-merge is preferred to over-merge throughout, so 317 and every count derived from it is an upper bound on the number of forks rather than a census, and §S7 gives the spread it carries.

S13. Case-Study Decisions

Charting a garden is an analysis with its own forks. main text §5.1 and main text §7.2 refer to the record of those decisions; Table S17 is that record, read from `policy.json`. The file was reconstructed after the study was charted, so each entry carries its provenance: *decided* where a person chose the value and the reason is on file, *implicit* where the value is a default nobody discussed. The implicit entries are the ones most worth revisiting, and the file lists them under `_open`.

Table S17. The case-study decisions. Read from `policy.json`, filled 2026-08-30 by the study owner with the agent proposing defaults. The provenance column is the file's own; the reason column condenses its `_provenance` notes.

decision	value	provenance	reason on file
artifacts charted	code; prose for silent decisions only	decided	the map rests on the code layer; prose spans reproduce at ARI 0.480 against code at 0.895 core
span comparison across builds	lenient, Jaccard ≥ 0.5 , 1-line tolerance	decided	the single largest lever; verbatim 0.176, lenient 0.713, core 0.895 on one extraction; the agent led with verbatim and was corrected
spans may be disjoint / interleaved	yes / no	implicit	the extraction prompt permits gaps and not interleaving; never argued
same-run repeat span	5 lines	decided	76 of 96 same-run merge groups spanning more than five lines were over-merges on reading; enforced as V6
singleton options	kept, labelled <i>novel</i>	decided	64% of options are singletons; excluding them moves by-option fork agreement from 0.31 to 0.69, a change in the question
option matching across builds	Jaccard on labels, 0.5	implicit	the <code>match_options</code> default; governs every cross-build comparison and has had no sensitivity check
fork exclusivity	required; violations kept as a finding	decided	86% of runs hold two options at one fork, dominated by the skin-tone exposure fork at 158 of 223; V2 makes it an audited exhibit
pivotal criterion	coverage $\geq 25\%$ and effective options > 1.5	decided	18 forks clear coverage alone; 17 clear both (main text §5.4, §S9)
lifecycle stages	six DSLC stages, assigned at the fork, revised after repair	decided	
role of stability figures	diagnostic, not a target	decided	the load-bearing refusal: induction can be driven to ARI 1.0 by merging almost nothing; no tuning authorised until the human benchmark supplies an external criterion
replicates per parent	5, pre-committed	decided	4 df per parent; 2 would give 1, and an sd on 1 df is compatible with almost anything
blocking checks	none	decided	most checks carry no null and rank for attention; a gate would convert a weak signal into a hard decision
review queue length	60	implicit	V1 alone queues 561 pairs; 60 is a guess at what a person will read
minimum runs for a test	15; multiplicity by BH	decided	

S14. Visualizations: Reading the Viewer

main text §4.5 names the views of `figures/forkscape.html` and the main text's Figures 3 to 12 show most of them. This section is the reading guide: what each mark encodes, which artifact it is read from, and what the controls do. The page is generated by `forkscape.py` from artifacts the pipeline has already written and computes no statistic of its own, so every number printed in it is the value in an artifact named above. The current build has four tabs, The Garden, Pivotal Forks, Lifecycle and Corpus Viewer; the About tab of earlier builds, which listed the artifacts each view is drawn from, is retired and this document carries that material. Figs. S1 to S14 are captures of the current build at the headline vocabulary.

The Garden. The Garden (Fig. S1; main text Figure 5 shows a highway selected) maps the paths analyses take between the stages of the data science lifecycle onto one ring. Each *sector* is a lifecycle stage, its width proportional to the number of decisions recorded in that stage. Each *node* is a fork, its area proportional to the number of analyses that reached it; high-coverage forks sit inward and rare ones are pushed to the rim. Each *chord* is a highway: at least three analyses went from one fork to the other in order (`min_link` in `data/analysis/circle_...induced.r3.osplit.json`), and its thickness is proportional to how many. Light chords bow inward in proportion to how light they are, so a three-analysis thread no longer traces the same arc as a fifty-analysis highway between nearby nodes, and chords are painted lightest-first so heavy traffic lands on top. A node is filled if it sits on a highway and hollow if it does not. The ten highest-coverage forks are numbered by default (`--top`, adjustable from 0 to 40 in the page), and the number ties a node to its row in the list beside the ring.

Chords are orange or purple. Orange means no persona is associated with the highway. Purple means the highway carries a persona association surviving Benjamini–Hochberg correction: some persona's permutation p falls at or below 3.69×10^{-4} , the threshold at FDR 0.05 across all route $\times$ persona cells, recorded as `enrichment_threshold` in the same artifact. As §S7 reports, the omnibus persona result reproduces while the identity of any single enriched cell does not, so a purple chord marks scan output rather than a finding. A highway is one entry per unordered pair of forks;

clicking its chord opens both directions as tabs in the detail panel, with the persona table following the selected direction while the permutation p stays undirected and says so. The persona control is six toggles, the five agentic briefs and the human teams, all ticked by default, and all ticked renders identically to none ticked. Ticking a subset is a union, so the comparison it poses is *where did any of these go*; a ticked subset filters the forks it reached and redraws each chord at that subset's own traversal count, while node size stays corpus-wide. Further filters keep only forks on a highway or only forks with an attribution score.

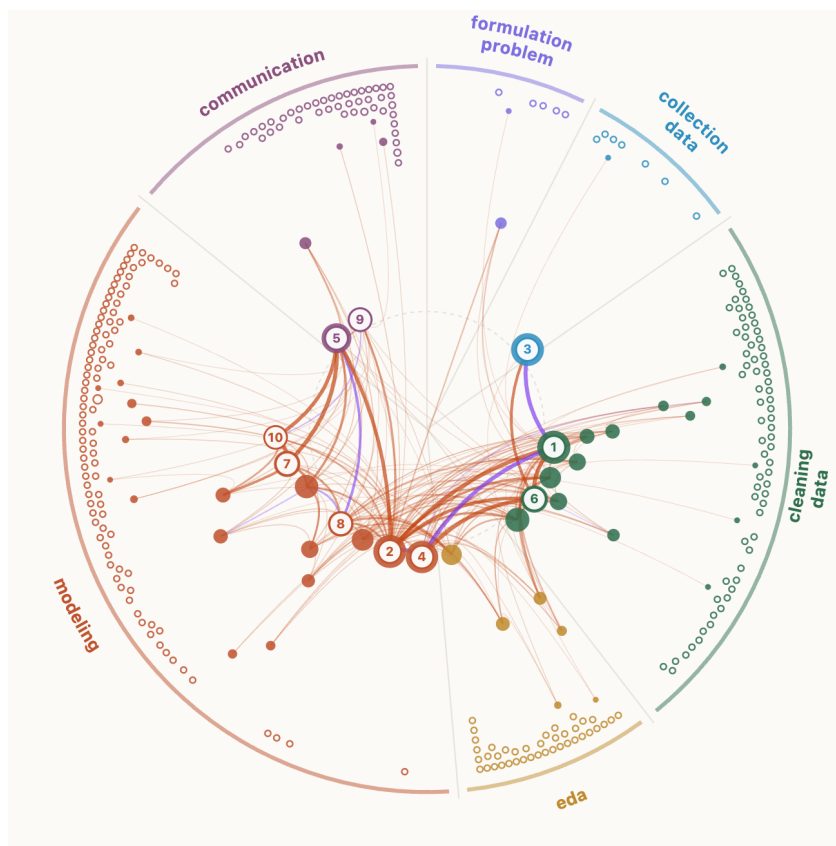


Fig. S1. The Garden. 317 forks on one ring, sectored by lifecycle stage, with all six persona toggles ticked. Node area is the number of analyses that reached the fork; chords are highways of three or more analyses, light ones bowed inward; the ten highest-coverage forks are numbered. Orange chords carry no persona association and purple ones carry an association that survives BH correction at $p \leq 3.69 \times 10^{-4}$.

Pivotal Forks. The tab draws the four panels of the fork atlas (figures/fork_atlas.svg) under either effective-option measure. `eff_all` is the pivotal criterion of §S9; `eff_reached` takes the shares over the analyses that reached the fork. The tab has toggles for the threshold, the coverage floor and which measure is in use, so a reader can see what the choice between them does. The *fan* (Fig. S2; main text Figure 8) ranks every fork by coverage on a log axis. The *map* (Fig. S3; main text Figure 11) plots coverage against effective options with the pivotal threshold as a dashed line. The *spectrum* (Fig. S4) draws the option shares of the top forks, and is where the difference between the two measures is visible: the hatched segment is the analyses that never reached the fork. The *by-stage* panel (Fig. S5) places each measurable fork by effective options within its lifecycle stage.

Attribution tables. The same tab carries the surrogate of §S9 as four tables, one per payout (Figs. S6 and S7; main text Figure 12 shows the verdict table). Each row is a player, meaning a fork that entered the design. The two middle columns are the fork's *strength*, drawn and printed: the mean over the 204 outcome-bearing analyses of the absolute Shapley value attributed to the fork, in the payout's units. For the verdict payout that is the average absolute shift in the probability of concluding *supported* that an analysis's choice at the fork produced; for $\log(\text{OR})$, the shift in log odds ratio; for z , the shift in z , the reported log odds ratio divided by its standard error; and for interval width, the shift in the log of the upper bound minus the log of the lower bound of the reported odds ratio. The last column is the permutation p : the proportion of refits in which the outcomes were permuted across analyses while decision profiles were held fixed and the



Fig. S2. The fan. Every fork, ranked left to right by coverage, on a log axis. A bar is coloured by lifecycle stage when the fork is pivotal under the selected measure and grey otherwise, so the long grey tail is the forks that are settled or too rarely reached to be contested.

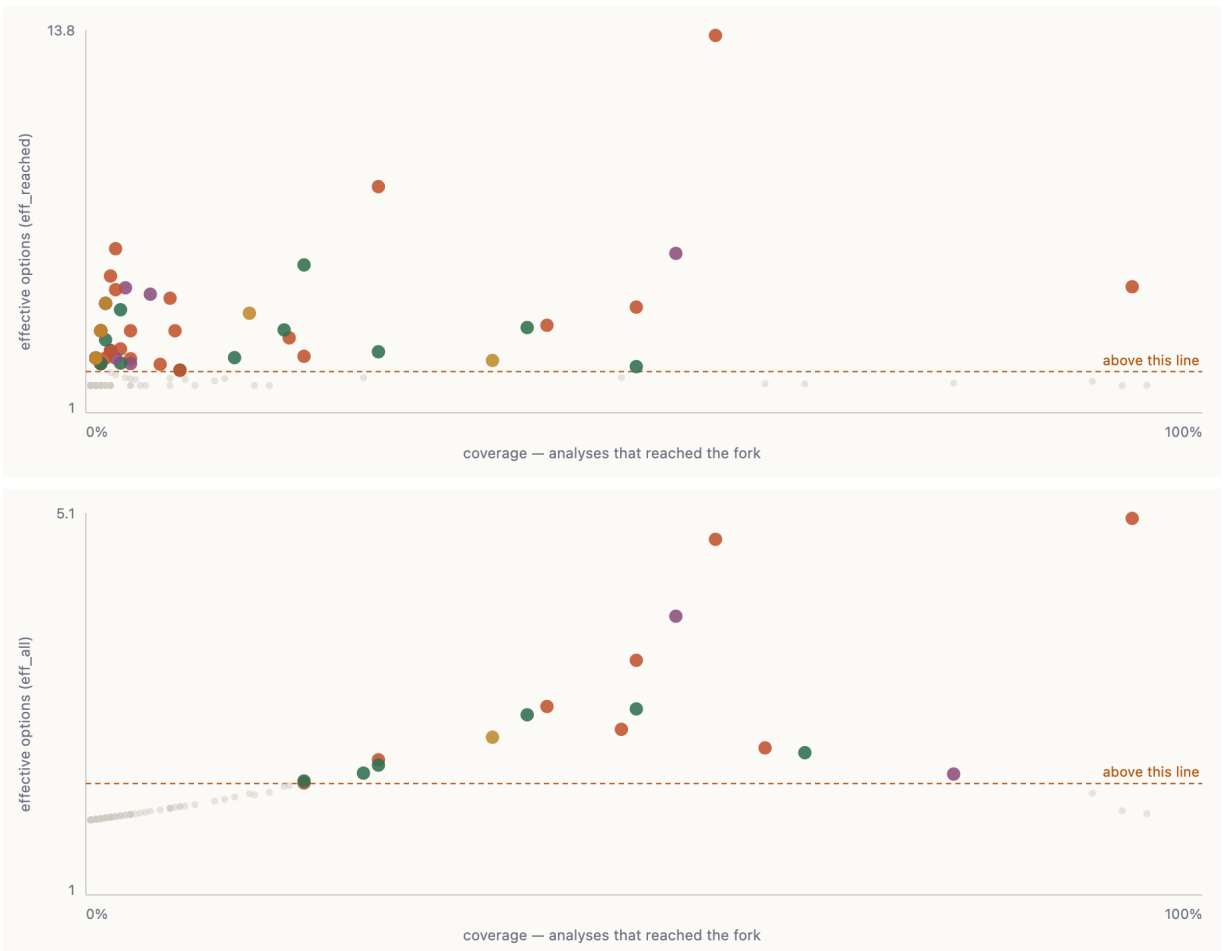


Fig. S3. The map. Coverage against effective options, one point per measurable fork, coloured by stage. *Top:* `eff_reached`. *Bottom:* `eff_all`. The dashed orange line is the pivotal threshold. Under `eff_all` the axis is bounded by coverage, since a fork most analyses never reach cannot be contested by them, which is why the two panels order the forks differently.



Fig. S4. The spectrum. Option shares of the top forks, largest option first. A solid segment is an option taken by several analyses, its shade lightening as the count falls; a pale segment is an option taken by exactly one analysis, which the clustering retains rather than absorbs; a hatched segment is the analyses that did not reach the fork, the level that separates `eff_all` from `eff_reached`.

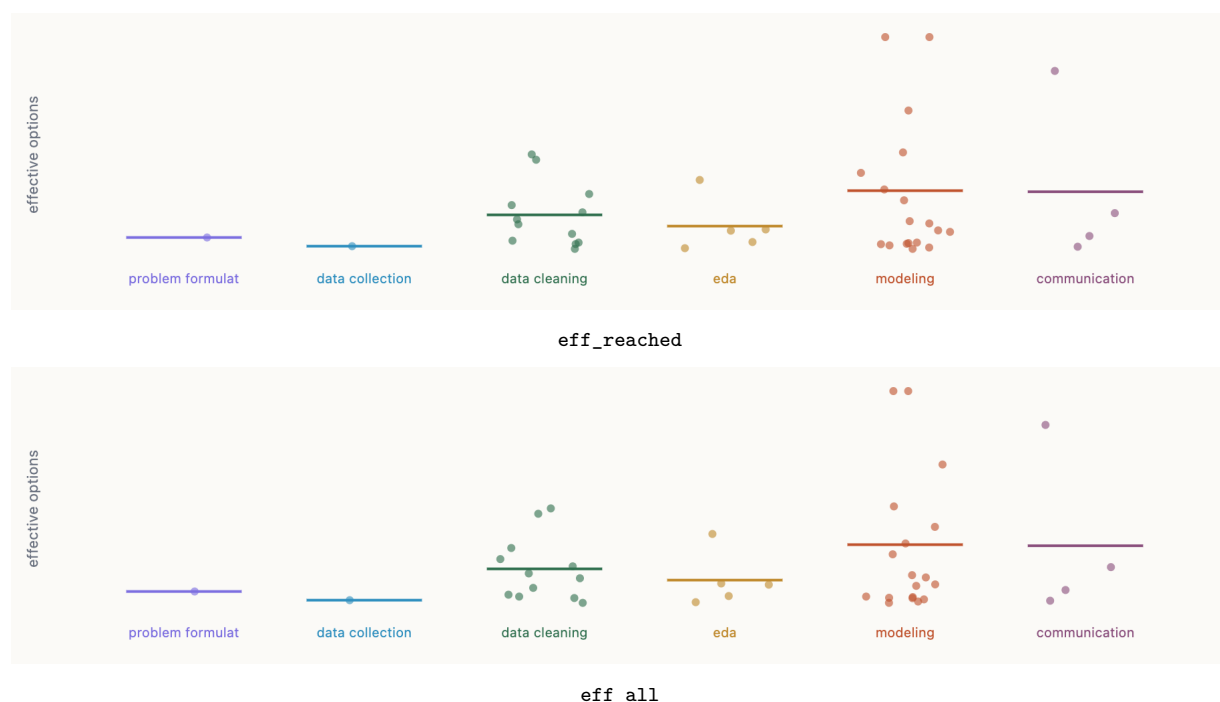
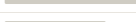


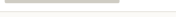
Fig. S5. Effective options by stage. Six lifecycle stages; each dot is a measurable fork placed vertically by its effective options, with the bar at the stage median. *Top: eff_reached. Bottom: eff_all.* Under either measure the contested forks sit in cleaning, modeling and communication, and problem formulation and data collection are settled.

fork's strength equalled or exceeded the observed one. A green bar marks $p < 0.05$ and grey a fork whose strength is not distinguishable from the label-shuffle null. The header of each table prints n , the in-sample R^2 and the cross-validated R^2 of Table S13. The bootstrap interval is not shown, for the reason given in §S9.

Lifecycle. The tab draws the iteration analysis of check set I over the six stages of the data science life cycle [39, 40]. It opens with two tables (Fig. S8): *where the decisions are*, the number of forks and of raw decision points in each stage, and *does the process go back*, the count of analyses with at least one return, the returns over all transitions, and the stage-to-stage returns ranked by count. Three charts follow. The *pipeline histogram* (Fig. S9; main text Figure 9) gives the share of decisions at each stage by position within the script, with every script's line numbers normalised by its own length and divided into 24 equal bins (`iteration.py:BINS`); stage was assigned blind to position (§S4), so the ordering is recovered rather than imposed. The *return-order heatmap* (Fig. S10; main text Figure 10) counts every stage-to-stage transition in the corpus, from the stage on the row to the stage on the column; cells above the diagonal follow the intended order and

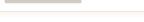
verdict · n=204 · R ² 0.486 · cross-validated R ² 0.2934			
how are the regression model's standard errors estimated?		+0.1146	p 0.000
which regression model, estimator, and SE scheme specifies the primary c		+0.0405	p 0.018
How are dyads with a missing skin-tone rating handled?		+0.0396	p 0.037
how is the skin-tone rating entered as the exposure — continuous or spli		+0.0369	p 0.130
what rule determines whether the effect is declared statistically suppor		+0.0275	p 0.185
how is complete-case/listwise deletion applied and over which variable s		+0.0268	p 0.052
How is the middle (Medium) skin-tone category handled?		+0.0251	p 0.003
how is the crude, unadjusted association estimated and tested?		+0.0242	p 0.165
how many categories, and by what mapping, is player position collapsed i		+0.0210	p 0.240
which covariates make up the model's adjustment set?		+0.0204	p 0.407
how are records with missing covariates handled — complete-case, impute,		+0.0201	p 0.315
When forming adjusted counterfactual predictions, are covariates average		+0.0189	p 0.345

verdict

log_or · n=204 · R ² 0.289 · cross-validated R ² 0.0808			
how are the regression model's standard errors estimated?		+0.0084	p 0.007
how is the skin-tone rating entered as the exposure — continuous or spli		+0.0037	p 0.393
are goalkeepers included in or excluded from the analysis sample?		+0.0037	p 0.015
how many categories, and by what mapping, is player position collapsed i		+0.0035	p 0.215
what rule determines whether the effect is declared statistically suppor		+0.0032	p 0.398
how are records with missing covariates handled — complete-case, impute,		+0.0032	p 0.297
Is the adjustment set expanded by adding covariates as a robustness chec		+0.0031	p 0.000
how is the crude, unadjusted association estimated and tested?		+0.0030	p 0.280
how are the confidence interval and p-value for the exposure effect comp		+0.0029	p 0.480
which covariates make up the model's adjustment set?		+0.0029	p 0.388
how is complete-case/listwise deletion applied and over which variable s		+0.0024	p 0.297
Are categorical predictors parameterized by dropping a reference level a		+0.0023	p 0.180

log odds ratio

Fig. S6. Attribution tables, verdict and log odds ratio payouts. Players sorted by strength, the mean absolute Shapley value over 204 analyses in the payout's units, with the permutation p at right; green marks $p < 0.05$. Read from data/analysis/shapley_...induced.r3.osplit.json and the H2 block of data/audits/shapley_audit_...induced.r3.osplit.json.

z · n=204 · R ² 0.569 · cross-validated R ² 0.3072			
how are the regression model's standard errors estimated?		+0.2091	p 0.000
which covariates make up the model's adjustment set?		+0.0778	p 0.105
how is the crude, unadjusted association estimated and tested?		+0.0650	p 0.135
how is the skin-tone rating entered as the exposure — continuous or spli		+0.0643	p 0.395
How are dyads with a missing skin-tone rating handled?		+0.0628	p 0.122
how many categories, and by what mapping, is player position collapsed i		+0.0611	p 0.182
how are records with missing covariates handled — complete-case, impute,		+0.0588	p 0.280
Is the adjustment set expanded by adding covariates as a robustness chec		+0.0544	p 0.003
are goalkeepers included in or excluded from the analysis sample?		+0.0490	p 0.028
How is the middle (Medium) skin-tone category handled?		+0.0449	p 0.048
how is complete-case/listwise deletion applied and over which variable s		+0.0445	p 0.253
which regression model, estimator, and SE scheme specifies the primary c		+0.0383	p 0.547

z statistic

ci_width · n=204 · R ² 0.146 · cross-validated R ² 0.0381			
how are the regression model's standard errors estimated?		+0.0061	p 0.007
how is the skin-tone rating entered as the exposure — continuous or spli		+0.0038	p 0.080
what rule determines whether the effect is declared statistically suppor		+0.0024	p 0.188
how are the confidence interval and p-value for the exposure effect comp		+0.0022	p 0.152
How are dyads with a missing skin-tone rating handled?		+0.0021	p 0.195
How is the middle (Medium) skin-tone category handled?		+0.0015	p 0.007
how is complete-case/listwise deletion applied and over which variable s		+0.0014	p 0.275
how are continuous predictors transformed and z-scored/standardized befo		+0.0011	p 0.380
which regression model and outcome family estimates the exposure-red-car		+0.0011	p 0.090
what bootstrap resampling scheme is used to compute the confidence inter		+0.0011	p 0.235
how is the crude, unadjusted association estimated and tested?		+0.0010	p 0.565
When forming adjusted counterfactual predictions, are covariates average		+0.0010	p 0.598

confidence-interval width

Fig. S7. Attribution tables, z and interval-width payouts. As Fig. S6. The standard-error fork leads every payout and survives the null on all four.

cells below it are returns to an earlier stage. The *return rates* chart (Fig. S11) lists the forks after which analyses most often return to an earlier stage, as the share of analyses reaching the fork that then went back, against the corpus rate of 0.19 (694 returns over 3,723 transitions, `return_rate` in `data/analysis/iteration_...induced.r3.osplit.json`). The measurement reads the delivered script rather than the process, so a script reorganised before saving conceals iteration that occurred, and every return rate is a lower bound.

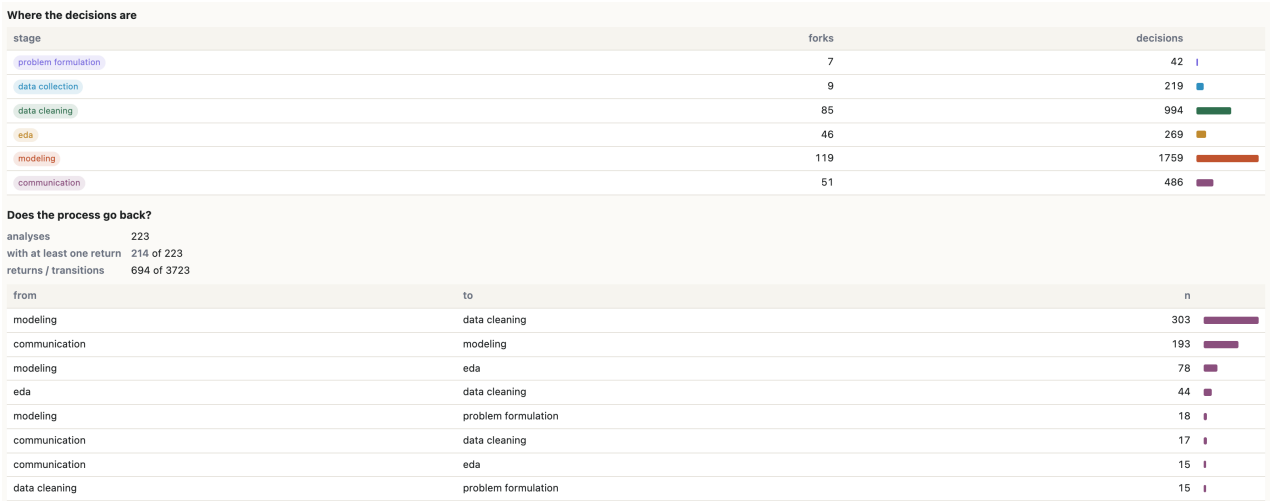


Fig. S8. Lifecycle tab, the two tables. *Top:* forks and raw decision points recorded in each stage. *Bottom:* 214 of 223 analyses return to an earlier stage at least once, 694 of 3,723 transitions are returns, and the largest single return is modeling back to data cleaning. Read from `data/analysis/iteration_...induced.r3.osplit.json`.

A · Pipeline histogram

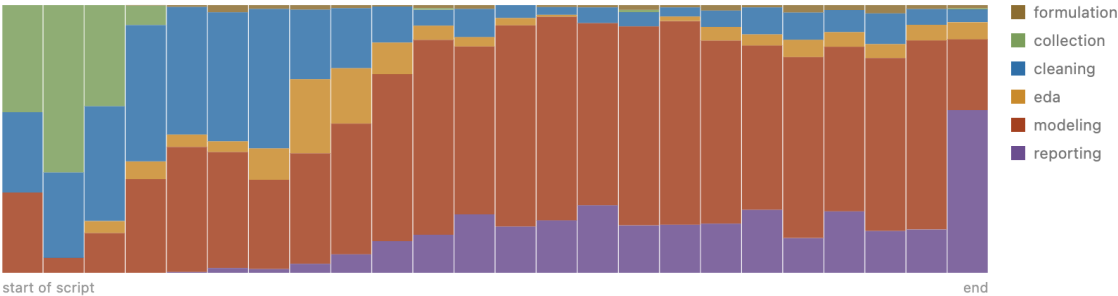


Fig. S9. Pipeline histogram. Share of decisions at each lifecycle stage by position within the script, over 223 analyses and 3,946 decisions, in 24 position bins from the start of the script to the end. Stages overlap everywhere: no bin is pure, and cleaning is still running in the final tenth.

Novelty. The same tab asks whether inventing an option moves the answer. An analysis's *novelty rate* is the share of its decisions that were options nobody else in the corpus chose. Fig. S12 (top) bins the 204 outcome-bearing analyses into five groups by novelty rate, of 40, 40, 40, 40 and 44 analyses, and plots the mean reported log odds ratio in each with a bootstrap 95% interval; a flat line would mean novelty does not move the answer. Fig. S12 (bottom) asks whether it matters *where* the novelty sits: the script is divided into five equal bands and the bar is the Spearman correlation across analyses between the novelty rate within that band and the reported log odds ratio. The five bands run -0.06 , $+0.06$, $+0.14$, $+0.17$ and -0.04 , so the association is largest in the third and fourth fifths and near zero at both ends; over the whole script the correlation with $\log(\text{OR})$ is Pearson 0.137 and with the binary verdict 0.242. Table S18 reports novelty entered as an additional player in the surrogate. It is significant on the two effect-size payouts and not on the verdict or z , adding most where the base model was weakest, which is the pattern expected if novelty acts on the magnitude of the estimate rather than on the conclusion.

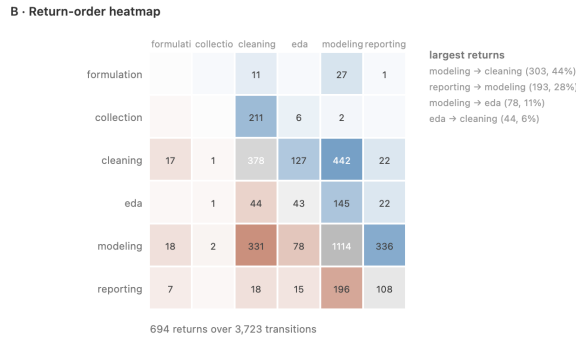


Fig. S10. Return-order heatmap. Number of transitions from the stage on the row to the stage on the column, summed over all analyses. Blue cells above the diagonal follow the lifecycle order; red cells below it are returns. The 694 returns are 44% of what random ordering would give (check I2).

C · Return rates of specific forks



Fig. S11. Return rates of specific forks. Forks with at least 15 opportunities, ranked by the share of analyses that returned to an earlier stage after reaching them, with the count in parentheses. The dotted line is the corpus rate of 0.19. The effect-size fork sends back 83% of the analyses that reach it, the signature of a result being computed and the model then being revisited.

Table S18. Novelty as an additional attribution player. Read from `data/analysis/shapley_...induced.r3.osplit.novelty.json` and the H2 block of the corresponding audit. *CV without* is the `.r3.osplit` column of Table S13 and *CV with* its `+novelty` column; gain is their difference. Strength is the novelty player's mean absolute Shapley value in the payout's units and *p* the share of label-shuffled refits in which it equalled or exceeded the observed one.

payout	CV without	CV with	gain	strength	p
verdict	0.293	0.297	+0.004	0.0231	0.255
log(OR)	0.081	0.114	+0.034	0.0065	0.000
z	0.307	0.320	+0.013	0.0690	0.163
CI width	0.038	0.082	+0.044	0.0101	0.000

Corpus Viewer. The Corpus Viewer (Fig. S13; main text Figure 3) lists every distilled run, one row each: 234 rows, the 204 agentic runs and the 30 human teams that were distilled, including the 11 whose write-up was segmented but which contribute no decisions. *run* names the analysis; *persona* is one of the five agentic briefs or *human*; *lines* is the length of its script; *% exon* is the share of those lines that do analytical work, and the *exon/intron* bar draws that partition along the script; *decisions* is how many raw decision points the analysis recorded and *silent* how many of those its write-up never mentions; *OR* is the odds ratio it reported and *conclusion* its verdict, with a dash for human teams whose outcome has not been extracted. Clicking a row opens the analysis (Fig. S14; main text Figure 4): the exon/intron map with silent decisions in orange, its exon share and decision count as percentiles against the corpus, and the forks it visited, each with the number of analyses that reached the same fork.

SI References

Entries are numbered as in the reference list of the main text.

- [3] Martin Bertran, Riccardo Fogliato, and Zhiwei Steven Wu. 2026. Many AI Analysts, One Dataset: Navigating the Agentic Data Science Multiverse. *Proceedings of the National Academy of Sciences of the United States of America* 123, 29 (July 2026), e2606495123. doi:10.1073/pnas.2606495123
- [14] Orris C. Herfindahl. 1950. Concentration in the U.S. Steel Industry. Columbia University (1950). Unpublished doctoral dissertation.
- [16] Albert O. Hirschman. 1945. *National Power and the Structure of Foreign Trade*. University of California Press (1945).
- [17] Lawrence Hubert and Phipps Arabie. 1985. Comparing Partitions. *Journal of Classification* 2, 1 (1985), 193–218.
- [25] Nathan Mantel. 1967. The Detection of Disease Clustering and a Generalized Regression Approach. *Cancer Research* 27, 2 (1967), 209–220.
- [34] Lloyd S. Shapley. 1953. A Value for n -Person Games. *Contributions to the Theory of Games* (1953), 307–317.
- [35] Raphael Silberzahn, Eric L. Uhlmann, Daniel P. Martin, et al. 2018. Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results. *Advances in Methods and Practices in Psychological Science* 1, 3 (2018), 337–356. doi:10.1177/2515245917747646
- [39] Bin Yu and Rebecca L. Barter. 2024. *Veridical Data Science: The Practice of Responsible Data Analysis and Decision Making*. MIT Press.
- [40] Bin Yu and Karl Kumbier. 2020. Veridical Data Science. *Proceedings of the National Academy of Sciences* 117, 8 (2020), 3920–3929. doi:10.1073/pnas.1901326117